

```

import numpy as np
import pandas as pd
import re
import pickle

import matplotlib.pyplot as plt
import seaborn as sns

from wordcloud import WordCloud, STOPWORDS

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, roc_curve
)

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

```

```

df = pd.read_csv(
    "https://raw.githubusercontent.com/Apaulgithub/oibsip_taskno4/main/spam.csv",
    encoding='ISO-8859-1'
)

df = df[['v1', 'v2']]
df.columns = ['Category', 'Message']
df['Spam'] = df['Category'].map({'ham': 0, 'spam': 1})

```

```

import nltk

nltk.download('stopwords')
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = re.sub(r'^a-zA-Z', ' ', text)
    text = text.lower().split()
    text = [lemmatizer.lemmatize(word) for word in text if word not in stop_words]
    return " ".join(text)

df['Message'] = df['Message'].apply(clean_text)

```

```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...

```

```

X = df['Message']
y = df['Spam']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```
models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression()
}

results = {}

for name, model in models.items():
    pipe = Pipeline([
        ('tfidf', TfidfVectorizer()),
        ('model', model)
    ])

    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    results[name] = accuracy_score(y_test, y_pred)
    print(f"{name} Accuracy:", results[name])
```

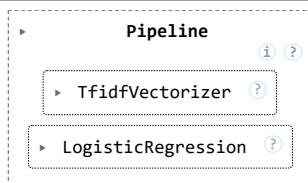
```
Naive Bayes Accuracy: 0.9632286995515695
Logistic Regression Accuracy: 0.95695067264574
```

```
best_model_name = max(results, key=results.get)
print("Best Model:", best_model_name)
```

```
Best Model: Naive Bayes
```

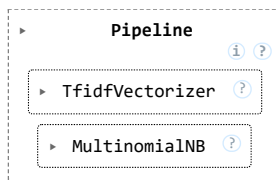
```
final_model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('model', LogisticRegression())
])

final_model.fit(X_train, y_train)
```



```
final_model = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('model', models[best_model_name])
])

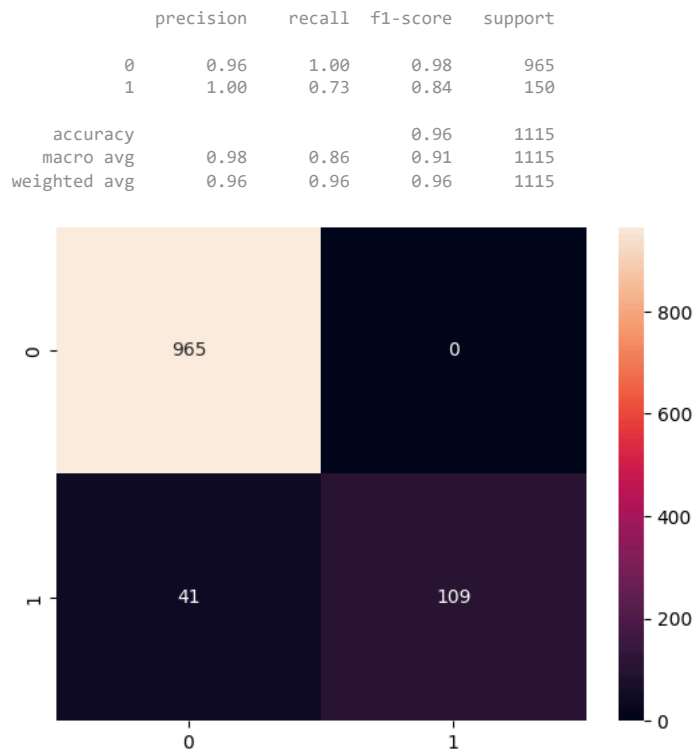
final_model.fit(X_train, y_train)
```



```
y_pred = final_model.predict(X_test)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```



```
import pickle

# Save model
with open('spam_model.pkl', 'wb') as f:
    pickle.dump(final_model, f)

print("Model saved successfully!")
```